Recursion & Problem Decomposition

Ibrahim El Mountasser
April 2, 2026

Plan

1. Recursion & Problem Decomposition
2. Example 1: Exponentiation
3. Example 2: Binary Search
4. Example 3: Counting Combinations
5. Divide and Conquer
6. How to Think About Recursion

Recursion & Problem Decomposition

Lecture: Recursion & Problem Decomposition

This lecture starts from a simple design question:

> *Given a word, how can we decide whether it is a palindrome?*

A first problem: palindrome

Take the word:

`racecar`

How would we check it?

A reasonable first move is to compare the two ends:

- first letter: `r`
- last letter: `r`

That part is easy.

Then what remains?

`aceca`

But this is not a new kind of problem. It is the *same* problem on a smaller input.

The pattern appears

After stripping matching outer letters, the question becomes:

> *Is the inside substring a palindrome?*

That smaller problem has exactly the same shape as the original one.

This is the moment where recursion appears naturally:

- solve the current problem by reducing it to a smaller copy of itself;
- keep reducing;
- stop when the instance becomes trivial.

That is the whole idea of recursion.

What every recursive solution needs

A recursive function needs two ingredients.

1. **Base case** A smallest instance that can be solved immediately.
2. **Recursive step** A rule that turns the current instance into a smaller instance of the same problem.

For palindrome checking:

- an empty string is a palindrome;
- a one-character string is also a palindrome;
- otherwise, compare the ends and recurse on the inside.

Palindrome: code after the idea

```
def is_palindrome(s):
    if len(s) <= 1:
        return True
    if s[0] != s[-1]:
        return False
    return is_palindrome(s[1:-1])
```

Read it as a mathematical statement:

- strings of size 0 or 1 are done;
- if the ends do not match, the answer is false;
- otherwise the answer depends entirely on the inside substring.

General recipe

When designing a recursive solution, ask:

1. What is the smallest version of the problem?
2. After doing one useful step, what smaller problem remains?
3. Is that smaller problem of the same type?
4. Does each step move us closer to the base case?

Good recursion is not magic. It is careful reduction.

Example 1: Exponentiation

Problem: compute $base^{exp}$

Suppose we want to compute:

$$2^5$$

One way is to think iteratively:

$$2 \cdot 2 \cdot 2 \cdot 2 \cdot 2$$

But there is another way to think.

If we already knew 2^4, then getting 2^5 would be easy:

$$2^5 = 2 \cdot 2^4$$

Now the structure looks familiar: the original problem reduces to the same problem with a smaller exponent.

Exponentiation

Once the decomposition is clear, the code is short.

```
def power(base, exp):
    if exp == 0:
        return 1
    return base * power(base, exp - 1)
```

Why this works:

- base case: $base^0 = 1$;
- recursive step: $base^{exp} = base \cdot base^{exp-1}$.

A better decomposition

The previous solution reduces the exponent by 1.
We can do better by asking a sharper question:

What if the exponent is even?

$$2^8 = (2^4)^2$$

More generally:

$$base^{exp} = \begin{cases} \left(base^{exp/2}\right)^2 & \text{if } exp \text{ is even} \\ base \cdot \left(base^{\lfloor exp/2 \rfloor}\right)^2 & \text{if } exp \text{ is odd} \end{cases}$$

Now the problem size is cut roughly in half each time.

Fast exponentiation

```
def fast_power(base, exp):
    if exp == 0:
        return 1

    half = fast_power(base, exp // 2)

    if exp % 2 == 0:
        return half * half
    return base * half * half
```

Same technique. Better reduction.

Example 2: Binary Search

Problem: find a value in a sorted list

Suppose we need to find 19 in:

$$[3, 6, 9, 12, 19, 25, 31]$$

Checking from left to right works, but wastes structure.
The list is sorted.

So the middle element tells us something important. If the middle is 12, then:

- everything to the left is ≤ 12;
- if the target is 19, the left half is useless.

We are left with the same search problem on half the list.

The smaller problem

After one comparison with the middle, exactly one of three things happens:

- the middle is the target;
- the target must be in the left half;
- the target must be in the right half.

Again, the structure is recursive:

$$\text{search current interval} \quad \longrightarrow \quad \text{search a smaller interval}$$

Binary search

```
def binary_search(a, target, left, right):
    if left > right:
        return -1

    mid = (left + right) // 2

    if a[mid] == target:
        return mid
    if target < a[mid]:
        return binary_search(a, target, left, mid - 1)
    return binary_search(a, target, mid + 1, right)
```

The base case is the empty interval:

$$left > right$$

Why binary search is fast

The important fact is not that recursion is used.
The important fact is how the problem shrinks:

$$n \to \frac{n}{2} \to \frac{n}{4} \to \frac{n}{8} \to \cdots$$

Each step removes half of the remaining search space.

This gives logarithmic time:

$$O(\log n)$$

The speed comes from the decomposition, not from recursion by itself.

Example 3: Counting Combinations

Problem: how many ways to choose k items from n?

Suppose we want the number of ways to choose k students from a group of n.
Direct counting gets messy quickly.
So focus on one particular student, say x.
Every valid group falls into one of two categories:

- groups that include x;
- groups that do not include x.

That split is clean, complete, and non-overlapping.

The recurrence emerges

Count the two cases separately.

If x is included:

$$\text{we still need to choose } k-1 \text{ from the remaining } n-1$$

If x is excluded:

$$\text{we still need to choose } k \text{ from the remaining } n-1$$

So:

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k}$$

That formula is not memorized first. It is discovered from the problem structure.

Choosing k from n

```
def choose(n, k):
    if k == 0 or k == n:
        return 1
    return choose(n - 1, k - 1) + choose(n - 1, k)
```

Base cases:

- choosing 0 items: exactly 1 way;
- choosing all n items: exactly 1 way.

This example matters

This is a deeper use of recursion than palindrome or exponentiation.
We are not just shrinking one value.
We are splitting the full problem into two smaller cases and combining their counts:

$$\text{answer} = \text{case } 1 + \text{case } 2$$

That pattern will return later in:

- divide and conquer;
- dynamic programming;
- backtracking.

Divide and Conquer

A different style: divide and conquer

Some recursive algorithms reduce to one smaller problem.
Others split into multiple substantial subproblems.
That second style is called **divide and conquer**:

1. divide the problem into smaller pieces;
2. solve each piece recursively;
3. combine the answers.

A classic example is sorting.

Problem: sort a list

Suppose we want to sort:

$$[8, 3, 5, 1, 7, 2]$$

Sorting the entire list directly is hard.
But a list of length 1 is already sorted.
So a natural idea is:

- split the list into two halves;
- sort each half;
- then merge the two sorted halves.

This is not a trick. It is the same design principle again: reduce the large problem to smaller instances of the same problem.

Merge sort

```
def merge_sort(a):
    if len(a) <= 1:
        return a

    mid = len(a) // 2
    left = merge_sort(a[:mid])
    right = merge_sort(a[mid:])

    return merge(left, right)
```

The merge step

```
def merge(left, right):
    result = []
    i = 0
    j = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

Why merge sort works

The recursive idea is simple:

- break the list until every piece is trivially sorted;
- rebuild the full answer by merging sorted pieces.

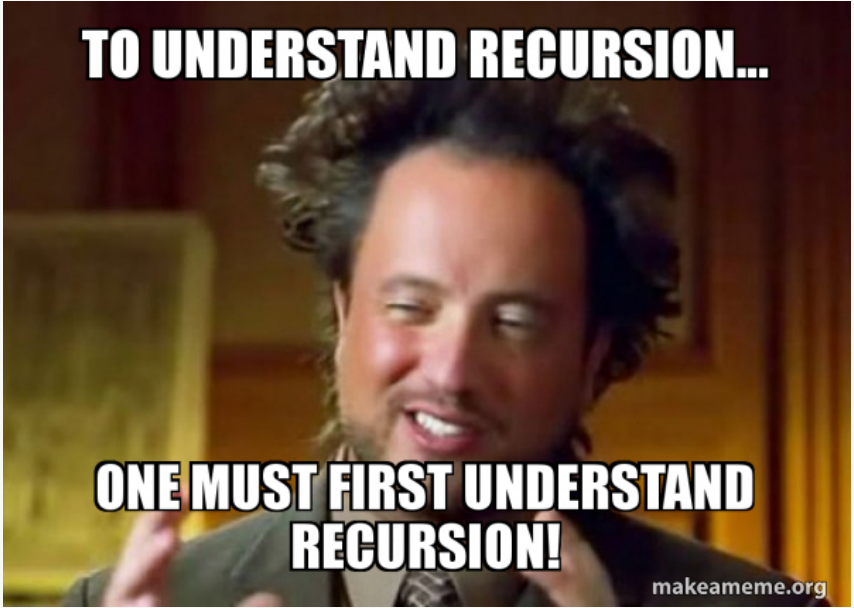
This is the divide-and-conquer pattern in full:

$$\text{divide} \rightarrow \text{conquer} \rightarrow \text{combine}$$

And because the splitting is balanced, merge sort runs in:

$$O(n \log n)$$

How to Think About Recursion


TO UNDERSTAND RECURSION...
ONE MUST FIRST UNDERSTAND RECURSION!
makeameme.org

A disciplined way to read recursive code

When reading a recursive function, do not panic and do not trace everything immediately. Look for four things:

1. the base case;
2. the smaller subproblem;
3. the measure that decreases;
4. the way the answer is combined.

If those four parts are clear, the function is usually clear.

Common mistakes

- No base case
- Base case exists but is never reached
- Recursive call does not make progress
- Wrong smaller problem
- Wrong combine step after the recursive call
- Using binary search on unsorted data

Most recursion bugs are decomposition bugs.

Summary

Today, recursion was introduced from actual problems:

- palindrome checking;
- exponentiation;
- binary search;
- counting combinations;
- merge sort as divide and conquer.

The central principle is this:

> *A recursive solution becomes natural once the problem is expressed as a smaller instance of itself.*